

CSE 4403: Algorithms

Lecture01

Introduction to Algorithms

Anika Farzana

Junior Lecturer

Department of Computer Science & Engineering

Islamic University of Technology, Dhaka, Bangladesh

Email: anikafarzana@iut-dhaka.edu

Origin of the Word

- Named after Al-Khwarizmi (c. 825 A.D.)
- Derived from the Latinized form of his name: *Algoritmi*
- Origin in mathematics
- Core concept in modern computer science

What is an Algorithm?

- A step-by-step procedure to solve a problem

What is an Algorithm?

- A step-by-step procedure to solve a problem
- Its like a recipe

What is an Algorithm?

- A step-by-step procedure to solve a problem
- Its like a recipe:
 - Takes inputs (data/ingredients),

What is an Algorithm?

- A step-by-step procedure to solve a problem
- Its like a recipe:
 - Takes inputs (data/ingredients),
 - Processes them through a finite number of steps, and

What is an Algorithm?

- A step-by-step procedure to solve a problem
- Its like a recipe:
 - Takes inputs (data/ingredients),
 - Processes them through a finite number of steps, and
 - Produces an output (result/finished dish).

What is an Algorithm?

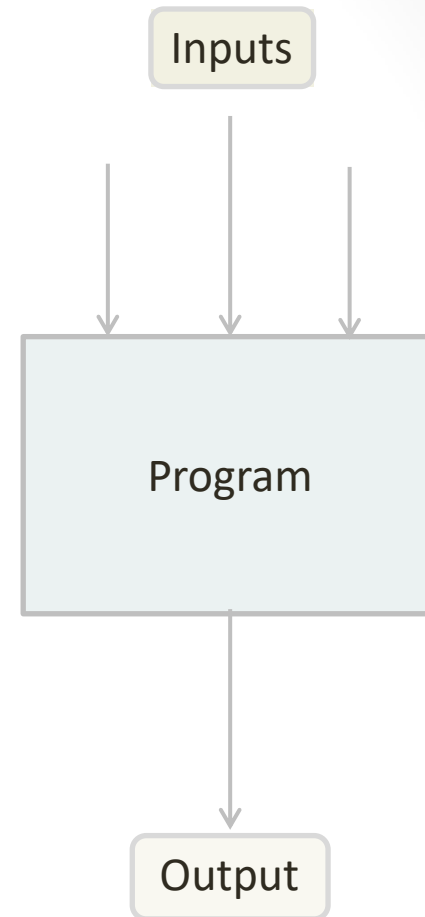
- A step-by-step procedure to solve a problem
- Its like a recipe:
 - Takes inputs (data/ingredients),
 - Processes them through a finite number of steps, and
 - Produces an output (result/finished dish).
- The problem must have some key properties

What is an Algorithm?

- A step-by-step procedure to solve a problem
- Its like a recipe:
 - Takes inputs (data/ingredients),
 - Processes them through a finite number of steps, and
 - Produces an output (result/finished dish).
- The problem must have some key properties
- Analyzed using time and space complexity

Key Properties

- Input: zero or more inputs
- Output: at least one output
- Definiteness: clear steps
- Finiteness: must terminate
- Effectiveness: simple operations



Key Properties

Definiteness

- Unambiguous steps
- Clear instructions
- E.g. No division by 0

Key Properties

Definiteness

- Unambiguous steps
- Clear instructions
- E.g. No division by 0

 Add 6 or 7 to x

 Compute $5/0$

Key Properties

Definiteness

- Unambiguous steps
- Clear instructions
- E.g. No division by 0

Finiteness

- Must stop
- Finite steps
- Efficiency matters

 Add 6 or 7 to x

 Compute $5/0$

Key Properties

Definiteness

- Unambiguous steps
- Clear instructions
- E.g. No division by 0

Finiteness

- Must stop
- Finite steps
- Efficiency matters

✗ Add 6 or 7 to x

✗ Compute $5/0$

✗ while True:

print("Hello")

Key Properties

Definiteness

- Unambiguous steps
- Clear instructions
- E.g. No division by 0

Finiteness

- Must stop
- Finite steps
- Efficiency matters

 Add 6 or 7 to x

 Compute $5/0$

 while True:
 print("Hello")

 while True:
 print("Hello")
 break

Key Properties

Definiteness

- Unambiguous steps
- Clear instructions
- E.g. No division by 0

Finiteness

- Must stop
- Finite steps
- Efficiency matters

Effectiveness

- Simple operations
- Feasible
- Practical

 Add 6 or 7 to x

 Compute $5/0$

 while True:
 print("Hello")

 while True:
 print("Hello")
 break

Key Properties

Definiteness

- Unambiguous steps
- Clear instructions
- E.g. No division by 0

- ✗ Add 6 or 7 to x
- ✗ Compute $5/0$

Finiteness

- Must stop
- Finite steps
- Efficiency matters

✗ while True:
 print("Hello")

✓ while True:
 print("Hello")
 break

Effectiveness

- Simple operations
- Feasible
- Practical

? Add two real numbers
and find the exact result

Key Properties

Definiteness

- Unambiguous steps
- Clear instructions
- E.g. No division by 0

Finiteness

- Must stop
- Finite steps
- Efficiency matters

Effectiveness

- Simple operations
- Feasible
- Practical

 Add 6 or 7 to x

 Compute $5/0$

 while True:
 print("Hello")

 while True:
 print("Hello")
 break

Add two real numbers
and find the exact result

$\sqrt{2} + \pi = ?$

$1.4142.. + 3.1416... = ?$

Key Properties

Definiteness

- Unambiguous steps
- Clear instructions
- E.g. No division by 0

- ✗ Add 6 or 7 to x
- ✗ Compute $5/0$

Finiteness

- Must stop
- Finite steps
- Efficiency matters

✗ while True:
 print("Hello")

✓ while True:
 print("Hello")
 break

Effectiveness

- Simple operations
- Feasible
- Practical

✗ Add two real numbers
and find the exact result
 $\sqrt{2} + \pi = ?$
 $1.4142.. + 3.1416... = ?$

Algorithm vs. Program

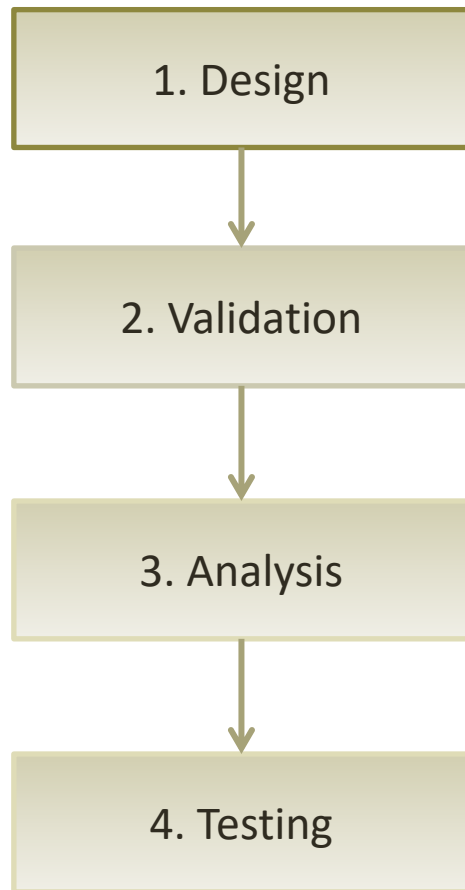
Algorithm

- Algorithm is the logic
- It is conceptual
- Natural language
- Universal time complexity

Program

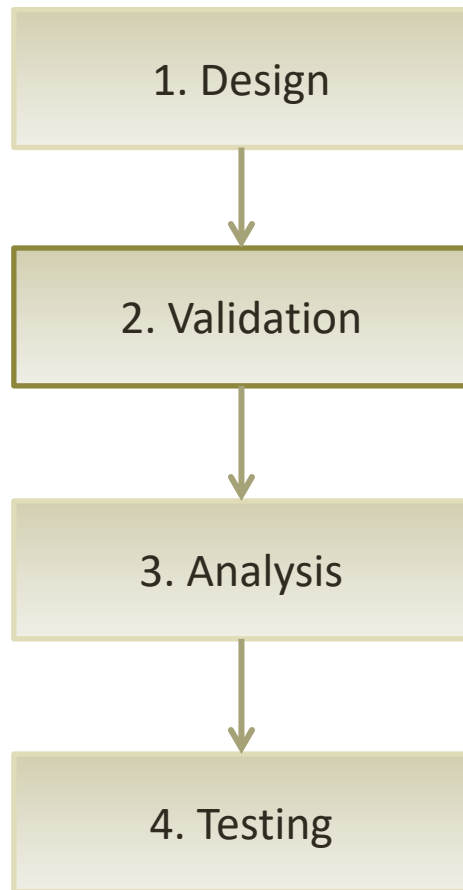
- Program is the implementation
- It is tangible
- Different languages
- Dependent on language

Building an Algorithm



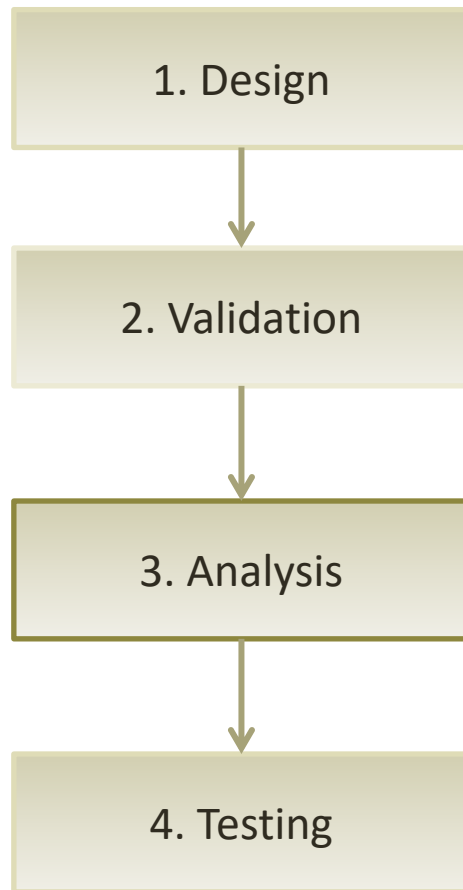
- ✓ Creative process
- ✓ Use known techniques
- ✓ Goal: efficient solutions

Building an Algorithm



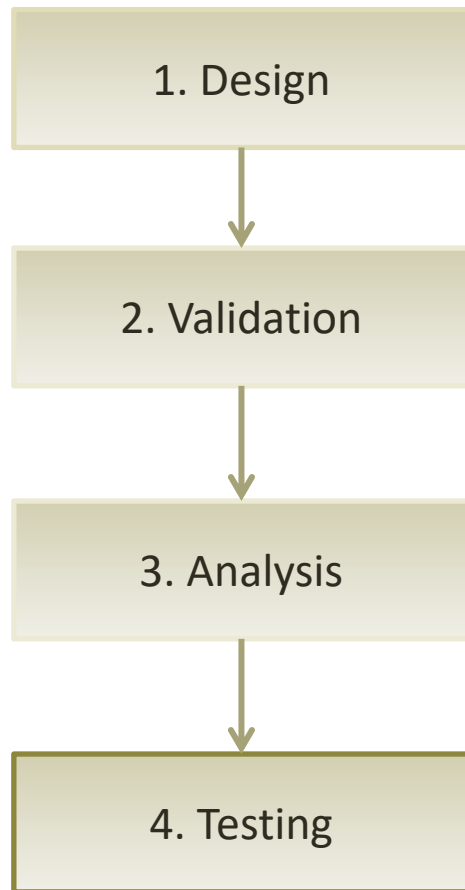
- ✓ Prove correctness
- ✓ Works for all inputs
- ✓ Independent of programming language

Building an Algorithm



- ✓ Time complexity
- ✓ Space complexity
- ✓ Compare efficiency

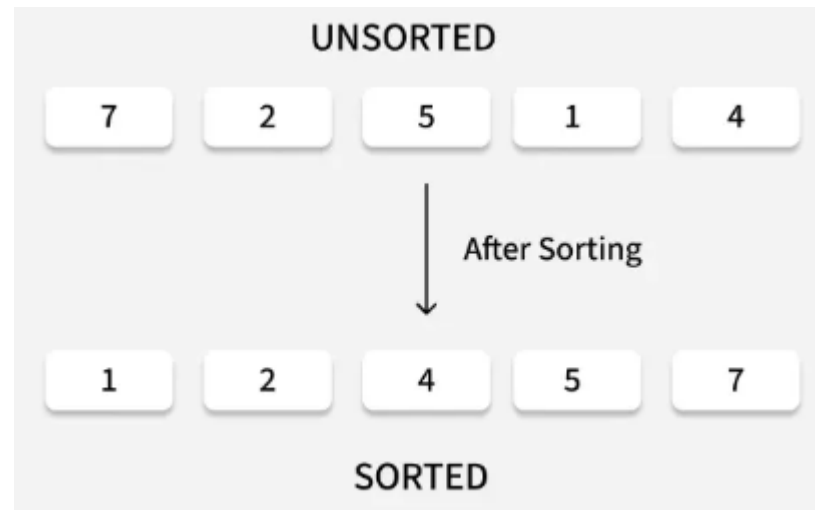
Building an Algorithm



- ✓ Debugging errors
- ✓ Edge cases
- ✓ Performance measurement

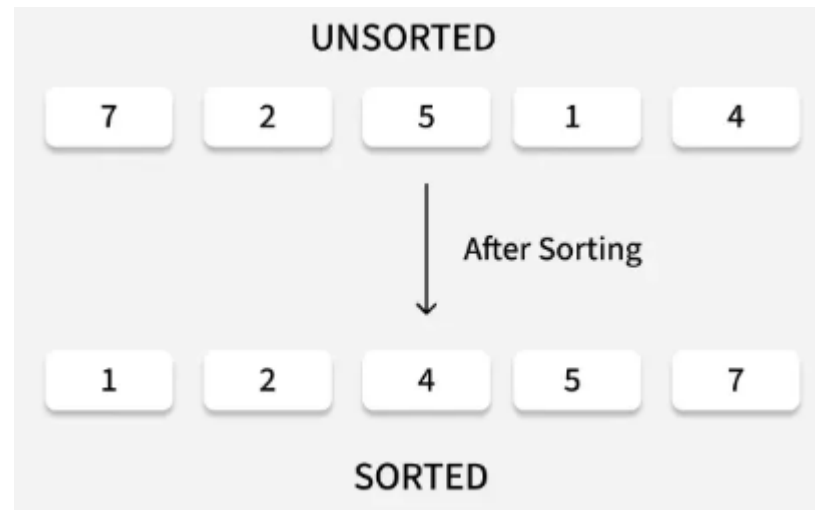
Sorting Algorithms

- Bubble Sort
- Insertion Sort
- Selection Sort
- Counting Sort
- Radix Sort



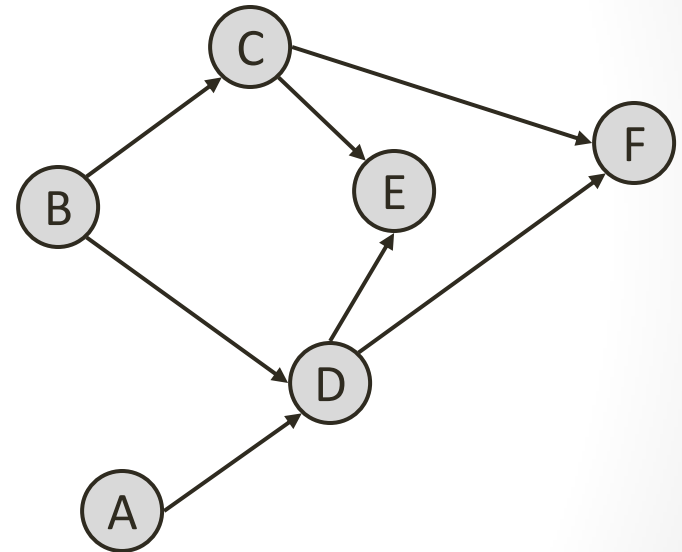
Sorting Algorithms

- Bubble Sort
 - Insertion Sort
 - Selection Sort
 - Counting Sort
 - Radix Sort
- ▶ MergeSort
 - ▶ QuickSort



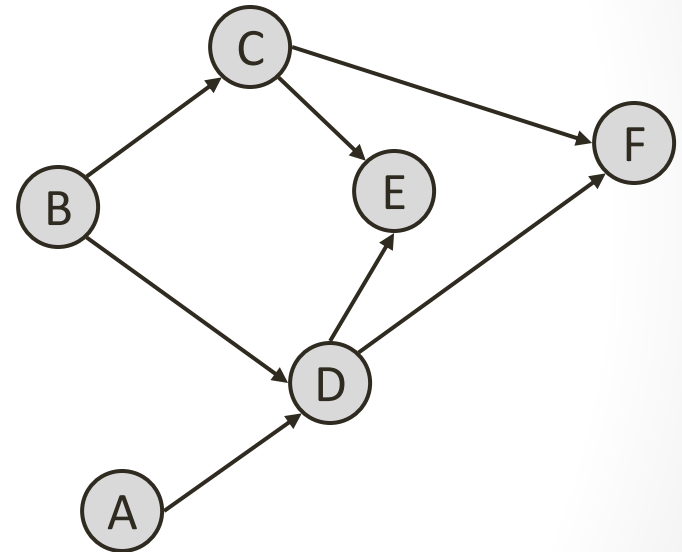
Graph Traversal

- Breadth-First Search (BFS)
- Depth-First Search (DFS)
- Topological Sort



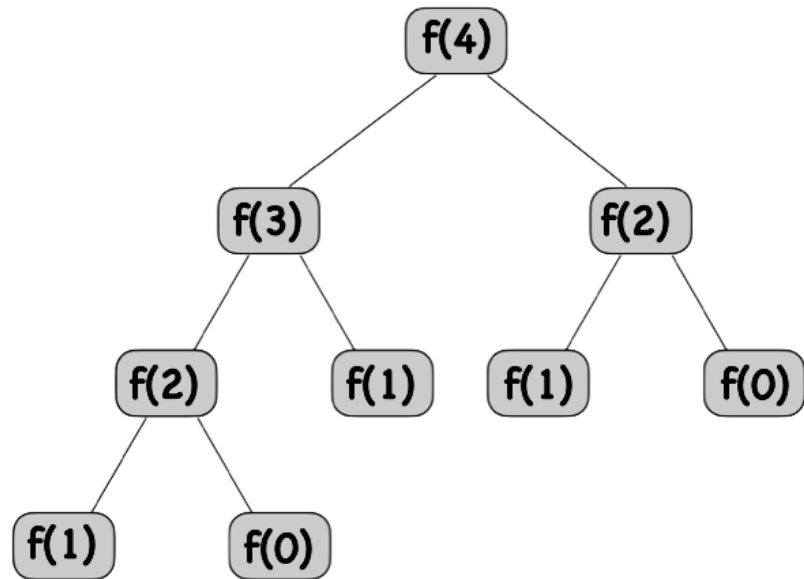
Shortest Path Algorithms

- Single Source Shortest Path
 - Dijkstra's Algorithm
 - Bellman-Ford Algorithm
- All Pair Shortest Path
 - Floyd Warshall



Dynamic Programming

- Fibonacci, Rod Cutting
- LCS, Edit Distance
- Knapsack, Coin Change
- Subset Sum



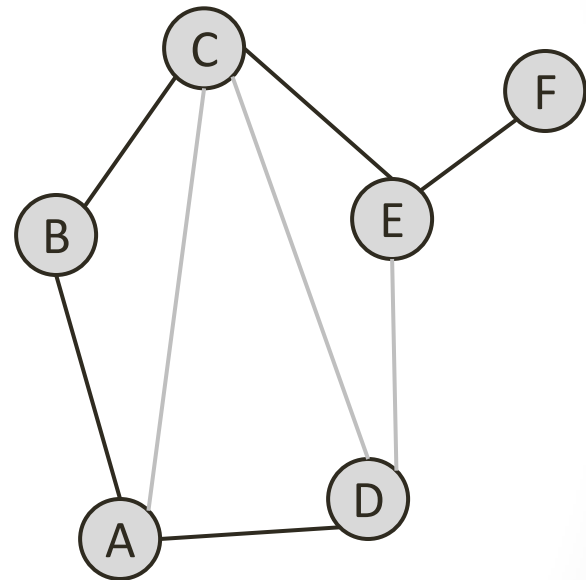
Dynamic Programming

- Bottom-up and Top-down
- Memoization

		Weights										
Number of items		0	1	2	3	4	5	6	7	8	9	10
	0	0	0	0	0	0	0	0	0	0	0	0
	1	0	0	0	0	6	6	6	6	6	6	6
	2	0	0	4	4	6	6	10	10	10	10	10
	3	0	0	4	5	6	9	10	11	11	15	15
	4	0	3	4	7	8	9	12	13	14	15	18
	5	0	3	4	7	8	9	12	13	14	16	18
	6	0	3	4	7	8	10	12	14	15	16	19

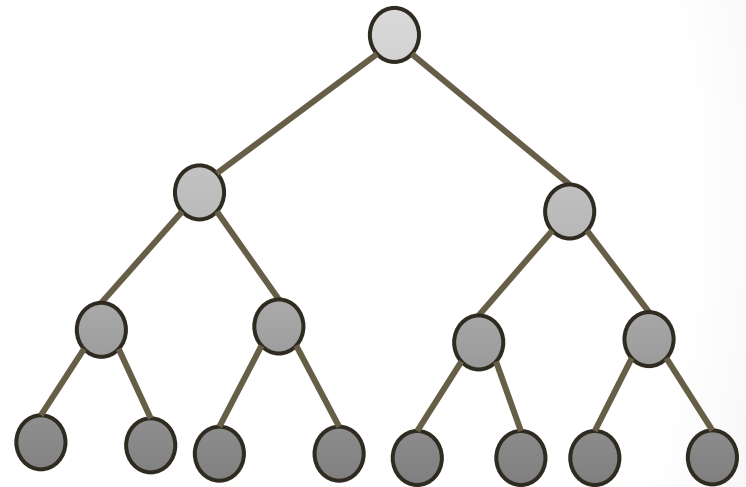
Advanced Topics

- Advanced DP & Greedy
- All-Pairs Shortest Paths
- Minimum Spanning Tree
 - Prims
 - Kruskals



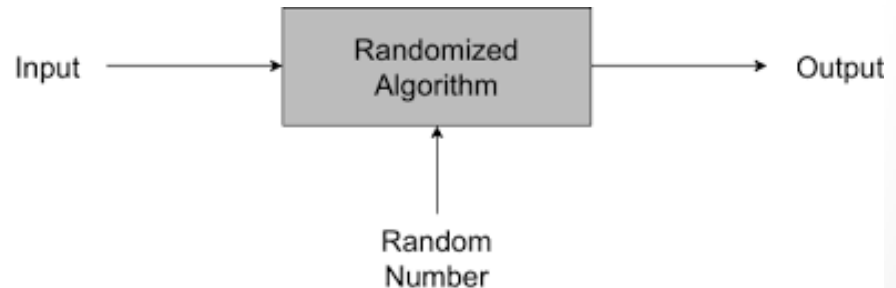
Divide & Conquer

- Recursion Trees
- MergeSort, QuickSort
- Interval Scheduling
- Convex Hull



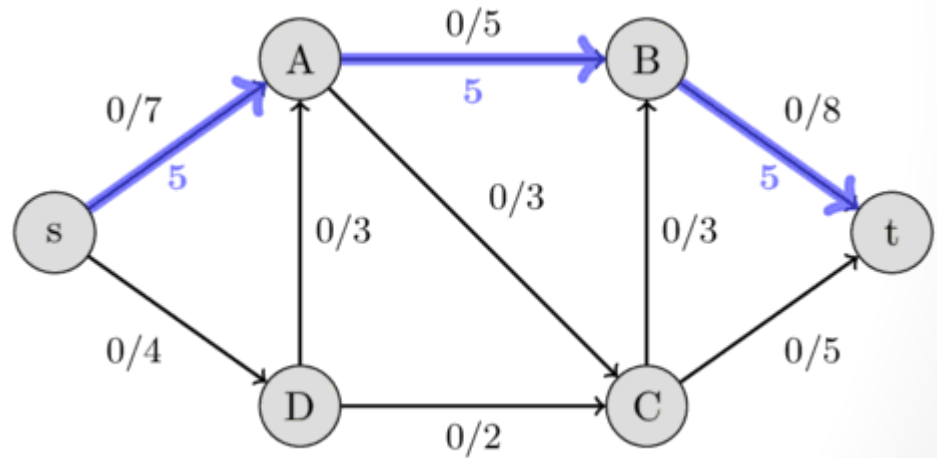
Randomized Algorithms

- Randomized Quick Sort
- Randomized Median
- Matrix Multiplication
- Hashing Techniques



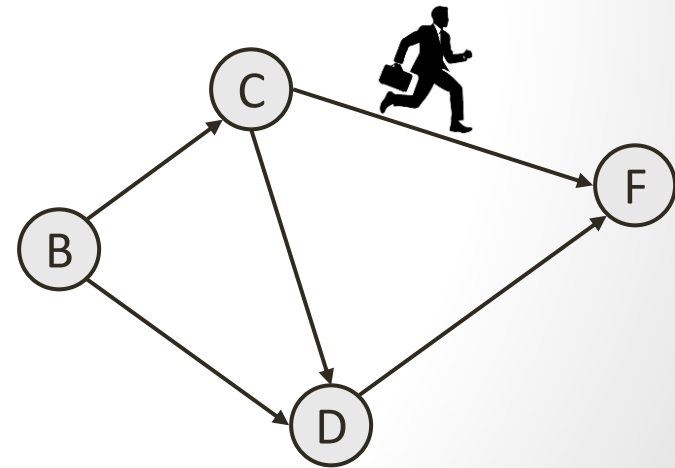
Incremental Improvement

- Max Flow & Min Cut
- Matching Algorithms



Computational Complexity

- P vs NP
- NP-Completeness
- Reductions
- Travelling Salesman Problem
- Approximation Algorithms



Books & References

- *Introduction to Algorithms*, 3rd edition by Thomas H. Cormen Charles E. Leiserson Ronald L. Rivest Clifford Stein
- *Computer Algorithms* by Ellis Horowitz, Sartaj Sahimi, Sanguthevar Rajasekaran